

Lab Report :Pulmonary Nodule Detection from CT Images Using Deep Learning (LUNA16 Dataset)

Nguyen Gia Nam

I. INTRODUCTION

Pulmonary nodules are small abnormal growths in the lungs that may indicate early-stage lung cancer. Detecting these nodules accurately is an important task in medical imaging because early diagnosis can significantly improve patient outcomes. However, pulmonary nodules vary in size, shape, and location, making automatic detection challenging.

In this lab work, I focused on volumetric (3D) detection of pulmonary nodules using computed tomography (CT) scans. Instead of analyzing individual 2D slices, the model processes the entire 3D CT volume, allowing it to learn spatial information across depth, height, and width. A 3D Convolutional Neural Network (3D CNN) was implemented to detect and localize nodules directly from the CT volume.

II. DATASET AND PREPROCESSING

A. Dataset Description

The dataset used in this work is the LUNA16 (Lung Nodule Analysis 2016) dataset, which is publicly available on Zenodo. LUNA16 is a curated subset of the LIDC-IDRI dataset and is widely used for lung nodule detection research.

For this experiment, I used:

- subset1 of the LUNA16 dataset
- The provided annotations.csv file

Each CT scan is stored using:

- A .mhd file (metadata: image size, spacing, orientation)
- A corresponding .raw file (actual voxel intensity data)

Each scan is a 3D volume with dimensions approximately:

- 512×512 pixels per slice
- 100–300 slices along the depth axis

The annotations.csv file contains:

- seriesuid: unique identifier for each CT scan
- coordX, coordY, coordZ: 3D coordinates of the nodule center
- diameter_mm: The diameter of the pulmonary nodule measured in millimeters.

Only scans from subset1 that appeared in annotations.csv were used, ensuring that every training sample contained a labeled pulmonary nodule.

B. Preprocessing Steps

Before feeding the data into the model, several preprocessing steps were applied:

- **CT Loading:** Each .mhd file was read using SimpleITK, which automatically links the .raw file.

- **Hounsfield Unit (HU) Clipping:** Voxel intensity values were clipped to the range $[-1000, 400]$ HU to remove irrelevant regions such as air and very dense tissues.
- **Normalization:** The clipped values were normalized to a range between 0 and 1 to stabilize training.
- **Tensor Conversion:** Each CT volume was converted into a PyTorch tensor with shape (1, Depth, Height, Width).

III. METHODOLOGY AND IMPLEMENTATION

I built a 3D Convolutional Neural Network (3D CNN) because it is well suited for volumetric medical images like CT scans. Unlike 2D CNNs, a 3D CNN can capture spatial features across multiple slices, which is critical for detecting pulmonary nodules.

The model includes:

- **3D Convolutional Layers:** These layers act as volumetric filters that learn spatial patterns of nodules across depth, height, and width. They help the model recognize round and blob-like structures typical of pulmonary nodules.
- **ReLU Activation:** ReLU introduces non-linearity, allowing the network to learn complex features.
- **MaxPooling3D:** Max pooling reduces the spatial size of the feature maps, helping the model focus on the most important regions while reducing computational cost.
- **Adaptive Average Pooling:** This layer converts feature maps of varying sizes into a fixed-length feature vector, making the network robust to different scan dimensions.
- **Fully Connected Layer:** The final layer predicts four continuous values: the x, y, z coordinates of the nodule center and its diameter.

This approach formulates pulmonary nodule detection as a regression problem, where the model directly predicts the nodule location and size from the CT volume.

A. Training Details

The model was trained using:

- **Optimizer:** Adam (adaptive learning rate, fast convergence)
- **Loss Function:** Mean Squared Error (MSE), suitable for coordinate regression
- **Epochs:** 30 (Enough for the model to gradually learn meaningful volumetric features)
- **Batch Size:** 1 (Due to the high memory cost of 3D CT volumes)
- **Validation Split:** 80% training, 20% validation

Training and validation loss were recorded at each epoch to monitor convergence and detect overfitting.

IV. RESULTS AND DISCUSSION

Figure 1 illustrates the training loss of the proposed model over 30 epochs. At the start of training, the loss is relatively high (around 34), which is expected since the model parameters are randomly initialized. During the first few epochs, the loss drops sharply, indicating that the model is quickly learning useful patterns from the input CT slices. This suggests that the data preprocessing steps, such as normalization and slice extraction, are effective and allow the model to learn efficiently.

After this initial phase, the loss continues to decrease more slowly and begins to stabilize around epochs 15 to 20, reaching values close to 12. This gradual reduction shows that the model is refining its predictions rather than making large updates. Small fluctuations in the loss curve are observed throughout training, which is expected because the model is trained with a very small batch size (batch size = 1). With such a batch size, each update depends heavily on a single image slice, leading to slight variations in the loss values.

The plateau observed after approximately 20 epochs suggests that the model has converged and that additional training does not lead to significant improvement. This behavior is reasonable given the limited dataset size and the fact that the model is trained on 2D slices instead of full 3D volumes. While a more complex architecture or 3D-based approach could potentially improve performance, such methods would require more data and higher computational resources.

model parameters are randomly initialized. The sharp decrease in loss during the first few epochs indicates that the model quickly captures basic patterns from the input data, confirming that the preprocessing steps and data pipeline are working correctly.

As training continues, the loss decreases more gradually and starts to stabilize around epochs 15 to 20. This behavior suggests that the model is no longer making large updates but instead refining its predictions. The small fluctuations observed in the loss curve are mainly caused by the use of a very small batch size (batch size = 1), where each update is influenced by a single image slice.

After approximately 20 epochs, the loss reaches a plateau, indicating that the model has converged and that further training does not significantly improve performance. This outcome is reasonable given the limited dataset size and the use of 2D slices rather than full 3D CT volumes. While more complex models could potentially achieve better results, they would require additional data and higher computational resources.

Overall, the training behavior demonstrates that the implemented model and training pipeline are stable and correctly configured. Although the approach is relatively simple, the consistent decrease and convergence of the training loss provide a solid baseline for future improvements and experimentation.

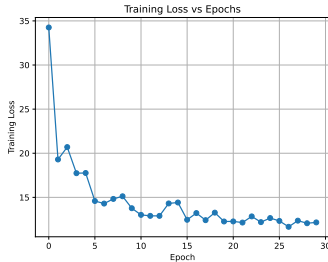


Fig. 1. Training Loss vs Epochs

Overall, the training results indicate that the implemented pipeline is working as intended. The model trains in a stable manner, the loss consistently decreases, and convergence is achieved without signs of instability. Although the performance is limited by dataset size and model simplicity, these results validate the correctness of the implementation and provide a solid baseline for future improvements, such as adding a validation set, experimenting with different architectures, or extending the approach to 3D data.

V. DISCUSSION

The training loss curve in Figure 1 shows that the model is able to learn effectively from the CT image slices. The high loss value at the beginning of training is expected because the